

I'll turn this into a concrete governance model: instruction precedence, repository harvesting, production-evidence capture, conflict resolution, iterative synthesis, token controls, and a conservative TOML upgrade that preserves existing rules.

The existing repository templates already establish global inheritance, Cursor/Codex mirroring, verification, and iteration notes. I'm extending that model rather than replacing it, with promotion gates so "self-learning" cannot turn one accidental implementation into a global default.

Thought for 1m 1s >

Analysis errored ^

</> Python



```
from pathlib import Path
import textwrap
import zipfile
import json

base = Path("/mnt/data/codex_orchestrator_bootstrap")
(base / ".codex" / "agents").mkdir(parents=True, exist_ok=True)

system_instructions = r"""# Codex Orchestrator Repository – Bootstrap and Operating Instructions

## 0. Execution mandate

You are operating inside the repository that will become the **central Codex/Cursor orchestration system.

Your task is to inspect the current machine-level instruction systems, preserve all existing instructions, and
synthesize a new, governed system.

1. Loads stable global instructions from:
   - `C:\Users\ppyxe\.codex`
   - `C:\Users\ppyxe\.cursor`
2. Loads repository-specific instructions and knowledge from the active repository.
3. Retrieves only the minimum relevant central knowledge for the current task.
4. Reuses production-proven workflows and implementations when they are compatible.
5. Learns from completed repositories and production outcomes.
6. Promotes durable lessons into central knowledge through explicit evidence and conflict resolution.
7. Publishes synchronized, non-destructive enhancements back to the Codex and Cursor master repositories.
8. Minimizes token use, credits, duplicated work, and unnecessary subagent fan-out.

This is a governed knowledge system, not an uncontrolled autonomous memory system.

---

## 1. Non-negotiable preservation rule
```

```
Existing files and rules in `C:\Users\ppyxe\.codex` and `C:\Users\ppyxe\.cursor` are auth

You MUST:

- Inventory them recursively before proposing changes.
- Read all instruction, rule, skill, agent, hook, TOML, JSON, YAML, Markdown, and MDC fil
- Preserve existing content, ordering, comments, paths, MCP definitions, authentication s
- Create timestamped backups before any write.
- Modify files through minimal, reviewable patches.
- Never replace an existing master file with a generic template.
- Never delete or rename an existing rule unless the user explicitly requests it.
- Never silently resolve a contradiction.
- Never write secrets, credentials, tokens, or machine-specific private data into the orc

When this bootstrap pack conflicts with an existing explicit user rule, preserve the user

Use this blocker format when safe merging is impossible:

```text
CRITICAL BLOCKER: Existing instruction/configuration cannot be safely merged because [exa
Required action: [specific user decision or missing input].
```

## 2. Instruction precedence

Apply instructions in this order, from highest to lowest authority:

1. Current explicit user instruction.
2. Safety, security, privacy, legal, and platform constraints.
3. Machine-level global instructions in `C:\Users\ppyxe\.codex` and `C:\Users\ppyxe\.cursor`.
4. Repository-root `AGENTS.override.md`, if present.
5. Repository-root `AGENTS.md`.
6. Nested directory `AGENTS.override.md` or `AGENTS.md` closest to the working directory.
7. Repository-specific skills and documented workflows.
8. Production-proven central knowledge that matches the current task and stack.
9. Repository-local historical knowledge and prior accepted decisions.
10. General defaults and model inference.

Rules closer to the active repository may specialize global rules but must not silently weaken security, privacy, verification, or explicit user constraints.

When two rules conflict:

- Prefer the more explicit rule.
- Prefer the narrower scope for repository-specific behavior.
- Prefer the newer approved rule when both have equal scope.

- Preserve both records and mark the superseded one.
  - Do not merge contradictory instructions into ambiguous prose.
- 

### 3. Repository role

This repository is the **orchestrator and knowledge control plane**.

It is responsible for:

- Registering source repositories.
- Incrementally inspecting changed repositories.
- Recording how successful implementations were reached.
- Capturing reusable workflows, decisions, failure modes, and verification commands.
- Distinguishing repository-local knowledge from globally reusable knowledge.
- Retrieving relevant knowledge for new tasks.
- Synthesizing candidate updates to global Codex/Cursor instructions.
- Publishing approved managed changes to both master folders.
- Measuring whether reused knowledge actually improved outcomes.

It is NOT responsible for:

- Becoming a copy of every source repository.
  - Storing source code that remains authoritative elsewhere.
  - Storing secrets, credentials, production payloads, PHI, personal data, or proprietary datasets.
  - Automatically declaring every deployed implementation a global best practice.
  - Overwriting repository-specific conventions with central preferences.
  - Creating recursive agent swarms.
  - Committing, pushing, deploying, or modifying production without explicit authority.
- 

### 4. Required architecture

Inspect the current repository first. Reuse existing structure where possible. If equivalent paths do not exist, create the following structure:

```
<orchestrator-root>/
├─ AGENTS.md
├─ README.md
├─ skills.md
├─ .codex/
│   └─ config.toml
│   └─ agents/
│       └─ workflow-router.toml
```



```

| | └─ repository-harvester.toml
| | └─ knowledge-synthesizer.toml
| | └─ verifier.toml
| └─ hooks/
└─ .cursor/
  └─ rules/
  └─ skills/
└─ config/
  └─ repositories.toml
  └─ promotion-policy.toml
  └─ retrieval-policy.toml
└─ master/
  └─ shared/
  └─ codex/
  └─ cursor/
└─ knowledge/
  └─ INDEX.md
  └─ catalog.jsonl
  └─ workflows/
  └─ patterns/
  └─ decisions/
  └─ failures/
  └─ commands/
  └─ pending/
  └─ deprecated/
  └─ schemas/
└─ state/
  └─ repository-snapshots.json
  └─ sync-runs/
  └─ retrieval-feedback.jsonl
└─ scripts/
└─ tests/
└─ logs/

```

## 4.1 Canonical versus deployed files

The version-controlled orchestrator repository is the canonical source for orchestrator-managed content.

The machine folders remain runtime targets:

```

C:\Users\ppyxe\.codex
C:\Users\ppyxe\.cursor

```



Do not mirror entire folders blindly.

Use a controlled publication model:

- Import existing master content.
- Separate user-managed content from orchestrator-managed content.

- Store orchestrator-managed source under `master/`.
- Publish only approved files or managed sections.
- Compare source and target before every write.
- Back up targets before publication.
- Produce a diff and validation report.

## 4.2 Managed content markers

For Markdown/MDC files, use bounded managed sections where practical:

```
<!-- BEGIN ORCHESTRATOR-MANAGED: <section-id> -->
...
<!-- END ORCHESTRATOR-MANAGED: <section-id> -->
```



Do not add duplicate managed blocks.

For TOML files:

- Do not use marker-based blind concatenation when it could duplicate keys.
- Parse or structurally inspect the file.
- Merge only allowlisted keys.
- Preserve existing unknown keys and comments.
- Reject duplicate-key output.
- Validate against the current Codex configuration schema.
- Back up the original file.

---

## 5. Knowledge taxonomy

Every knowledge record must have exactly one primary type:

- `workflow`: repeatable sequence that completes a task.
- `pattern`: reusable implementation or architecture pattern.
- `decision`: accepted choice with alternatives and rationale.
- `failure`: failure mode, diagnosis, and proven correction.
- `command`: canonical executable command or command sequence.
- `constraint`: compatibility, security, regulatory, or operational boundary.
- `preference`: stable user workflow or output preference.
- `reference`: authoritative documentation or source map.
- `anti_pattern`: approach that should not be repeated.

Repository-specific implementation details remain local unless promotion criteria are met.

---

## 6. Knowledge record schema

Create a machine-readable schema and enforce it.

Each record must contain:

 YAML 

```
id: stable-kebab-case-id
title: concise descriptive title
type: workflow|pattern|decision|failure|command|constraint|preference|reference|anti_patt
scope: global|domain|stack|repository
status: candidate|validated|production_proven|canonical|deprecated|rejected
summary: concise retrieval summary
trigger_phrases:
  - phrase
tags:
  - tag
applicability:
  languages: []
  frameworks: []
  runtimes: []
  operating_systems: []
  deployment_targets: []
  repository_markers: []
incompatible_when: []
procedure: []
verification: []
rollback: []
evidence:
  source_repository: ""
  source_commit: ""
  source_paths: []
  task_or_issue: ""
  tests_executed: []
  deployment_environment: ""
  deployment_evidence: ""
  accepted_by_user: false
  production_observation_window: ""
provenance:
  created_at: ISO-8601
  updated_at: ISO-8601
  created_by: ""
  supersedes: []
  derived_from: []
confidence:
  score: 0.0
  rationale: ""
reuse:
  retrieval_count: 0
  successful_reuse_count: 0
```

```
failed_reuse_count: 0
last_retrieved_at: null
```

No record may omit provenance, scope, status, or verification.

---

## 7. Knowledge lifecycle

Use the following states:

### 7.1 Candidate

Created after a completed task when a reusable lesson may exist.

A candidate is not a default.

### 7.2 Validated

The implementation or workflow:

- Passed its stated checks.
- Matches the actual repository state.
- Has explicit evidence.
- Was accepted by the user or merged through the normal repository process.

### 7.3 Production-proven

The implementation:

- Was deployed or used in the intended environment.
- Has a traceable release, deployment, or operational reference.
- Did not require rollback during the stated observation window.
- Has no known unresolved critical incident caused by the pattern.
- Retains its original applicability constraints.

“Present in a production branch” is not sufficient by itself.

### 7.4 Canonical

A production-proven record becomes a default only when at least one condition is met:

- It succeeded in two or more independent repositories.
- The user explicitly approves it as the preferred default.
- It is an exact stable preference or operating rule supplied by the user.
- It is a security or compatibility constraint backed by authoritative evidence.

### 7.5 Deprecated or rejected

Never delete history.

Move obsolete records to `knowledge/deprecated/` or mark them rejected.

Record:

- Replacement.
  - Reason.
  - Date.
  - Affected repositories.
  - Migration guidance.
- 

## 8. Production evidence hierarchy

Use this evidence order:

1. User-confirmed production success.
2. Deployment record plus successful post-deployment verification.
3. Release tag or immutable commit plus operational evidence.
4. CI/CD success plus environment-specific smoke tests.
5. Passing local integration tests.
6. Passing unit tests.
7. Code inspection only.
8. Agent inference only.

Levels 7 and 8 cannot independently justify `production_proven` or `canonical`.

---

## 9. Startup behavior for every task

Before editing code or spawning subagents:

1. Read active global instructions.
2. Read repository `AGENTS.md` and closest nested overrides.
3. Read the relevant repository workflow from `skills.md` or `.cursor/skills/`.
4. Inspect repository markers, manifests, lockfiles, deployment files, and tests.
5. Classify the task by:
  - domain,
  - language,
  - framework,
  - task type,
  - risk,
  - deployment target.
6. Query the central knowledge index using those attributes.
7. Retrieve a maximum of:

- 3 primary records,
  - 2 supporting records,
  - 1 relevant failure/anti-pattern.
8. Retrieve repository-local knowledge.
  9. Compare compatibility before reuse.
  10. State the intended workflow and verification route internally.
  11. Execute the smallest coherent change.
  12. Verify.
  13. Review the diff.
  14. Record outcomes and candidate lessons.

Do not load the entire knowledge base into context.

---

## 10. Retrieval policy

Central knowledge is a retrieval system, not a prompt dump.

### 10.1 Index-first retrieval

`knowledge/INDEX.md` and `knowledge/catalog.jsonl` must contain short summaries and metadata sufficient to shortlist records without opening full files.

### 10.2 Retrieval ranking

Rank records by:

1. Exact task-type match.
2. Exact stack/runtime match.
3. Repository-marker match.
4. Production-proven or canonical status.
5. Successful reuse history.
6. Recency, only when the underlying technology is time-sensitive.
7. User-approved preference.
8. Confidence score.

### 10.3 Compatibility gate

Before reusing a record, verify:

- Runtime and framework compatibility.
- Dependency/version compatibility.
- Operating-system compatibility.
- Deployment compatibility.
- Data and security constraints.
- Public interface compatibility.

- Repository-specific rules.

If compatibility is uncertain, treat the record as advisory, not authoritative.

## 10.4 Retrieval budget

Default retrieval budget:

```
Primary records: 3
Supporting records: 2
Failure records: 1
Maximum full records opened: 6
Maximum central-knowledge context: 12,000 tokens
```



Lower the budget for simple tasks.

Increase it only when the task is cross-cutting, high-risk, or the initial retrieval is insufficient.

---

## 11. Repository harvesting

Only harvest repositories listed in `config/repositories.toml`.

For each repository:

1. Resolve canonical path.
2. Confirm it is accessible.
3. Read its current commit or content fingerprint.
4. Compare with `state/repository-snapshots.json`.
5. Skip unchanged repositories.
6. Inspect only changed commits/files since the previous successful harvest.
7. Read:
  - repository instructions,
  - skills/workflows,
  - README and architecture docs,
  - manifests and lockfiles,
  - CI/CD configuration,
  - deployment configuration,
  - tests,
  - changelogs/releases,
  - accepted implementation files,
  - postmortems or failure notes.
8. Exclude:
  - secrets,

- `.env` content,
- credentials,
- generated caches,
- dependency directories,
- build artifacts,
- large raw datasets,
- production payloads,
- personal or regulated data.

9. Produce candidates, not immediate global rules.

10. Record repository fingerprint only after a successful harvest.

Do not infer production success solely from filenames or branch names.

---

## 12. Learning extraction after a task

At task completion, create a learning delta only when one or more of these occurred:

- A reusable workflow was established.
- A prior workflow failed and was corrected.
- A hidden dependency or environment constraint was discovered.
- A verification command was proven.
- A deployment-specific requirement was identified.
- A stable user preference was clarified.
- A central pattern was successfully reused.
- A central pattern failed due to a compatibility boundary.

The learning delta must answer:

1. What task was performed?
2. What approach was used?
3. What evidence shows it worked?
4. What failed before the final approach?
5. Which parts are reusable?
6. Which parts are repository-specific?
7. When should this not be used?
8. What verification proves success?
9. What rollback exists?
10. Should it remain local or become a promotion candidate?

Do not record routine facts that are already obvious from the source code.

---

## 13. Promotion policy

## 13.1 Automatic promotion allowed

Automatic promotion from `candidate` to `validated` is allowed only for low-risk records that have:

- Complete provenance.
- Passing verification.
- No conflicts.
- No secrets or sensitive data.
- Narrow applicability.
- Deterministic content.
- A successful source task.

Examples:

- Correct repository command.
- Stable path mapping.
- Exact formatter/test invocation.
- Documented non-sensitive environment prerequisite.
- Reproducible file-generation sequence.

## 13.2 Human approval required

Require explicit user approval before a record becomes global or canonical when it affects:

- Architecture.
- Dependency selection.
- Security.
- Authentication.
- Network access.
- Database migrations.
- Production deployment.
- Cost-bearing services.
- Data retention.
- Clinical, legal, financial, or regulated behavior.
- Destructive commands.
- Global output style.
- Agent permissions.
- Approval policy.
- Sandbox policy.
- MCP servers.
- Model/provider configuration.

## 13.3 Promotion score

Calculate a score for prioritization, not as the sole approval mechanism:



|                           |       |
|---------------------------|-------|
| evidence quality          | 0-30  |
| verification completeness | 0-20  |
| successful reuse          | 0-20  |
| scope clarity             | 0-10  |
| rollback quality          | 0-10  |
| recency/relevance         | 0-10  |
| total                     | 0-100 |

### Suggested thresholds:



|        |                                                         |
|--------|---------------------------------------------------------|
| < 50   | keep local or reject                                    |
| 50-69  | candidate                                               |
| 70-84  | validated                                               |
| 85-94  | production-proven candidate                             |
| 95-100 | canonical candidate, still subject to required approval |

---

## 14. Conflict resolution and deduplication

When a new candidate overlaps existing knowledge:

1. Compare scope.
2. Compare stack and version applicability.
3. Compare evidence strength.
4. Compare verification.
5. Compare successful reuse.
6. Compare recency only when technically relevant.
7. Decide one of:
  - merge,
  - specialize,
  - supersede,
  - reject,
  - retain both with explicit applicability boundaries.

Never flatten two stack-specific workflows into a vague global workflow.

Every merged or superseded record must retain links to source records.

---

## 15. Token and credit minimization

Use the following defaults:

- Index before full-file reads.
- Read changed files before unchanged files.

- Use Git diffs and content hashes.
- Reuse existing commands and scripts.
- Prefer deterministic local inspection over model inference.
- Do not restate the complete task to every subagent.
- Give subagents only their narrow scope and relevant files.
- Do not use multiple agents for a task one agent can complete reliably.
- Do not recursively delegate.
- Do not ask separate agents to produce overlapping summaries.
- Do not load complete repository histories unless necessary.
- Do not re-read files that have not changed during the same run.
- Cache repository metadata and retrieval results by commit hash.
- Keep knowledge summaries concise.
- Store detailed evidence in linked files, not in every index entry.
- Default to low or medium reasoning.
- Use a higher-reasoning profile only for architecture, complex debugging, security review, or conflict synthesis.
- Run targeted tests before full suites.
- Stop after sufficient evidence; do not keep exploring after the acceptance criteria are met.

---

## 16. Multi-agent policy

Subagents are optional specialized workers.

Default limits:

```
maximum concurrent threads: 3
maximum nesting depth: 1
```



Use a subagent only when work is independently separable.

Approved roles:

- `workflow_router`: classify task and retrieve relevant knowledge.
- `repository_harvester`: inspect source repositories read-only and create candidate records.
- `knowledge_synthesizer`: deduplicate and propose promotions.
- `verifier`: independently validate diffs, tests, evidence, and promotion eligibility.

The parent agent remains responsible for:

- Scope.

- Final decisions.
- Conflict resolution.
- Writes.
- User-facing claims.
- Verification completeness.

Do not let a harvester or verifier modify source repositories.

---

## 17. Global Codex/Cursor synchronization

### 17.1 Import phase

Before creating orchestrator-managed master files:

1. Inventory both global folders.
2. Hash all candidate files.
3. Copy non-sensitive instruction content into a dated import snapshot.
4. Identify semantic duplicates.
5. Identify contradictions.
6. Produce `reports/global-instruction-audit.md`.
7. Do not publish changes during the audit phase.

### 17.2 Synthesis phase

Build shared semantic rules under:

```
master/shared/
```



Build surface-specific renderings under:

```
master/codex/  
master/cursor/
```



Codex-specific content may include:

- `AGENTS.md`
- `config.toml` merge specification
- custom agent TOMLs
- skills

Cursor-specific content may include:

- `.mdc` rules
- Cursor skills
- tool/editor-specific paths

Shared semantics must remain aligned, but syntax may differ.

### 17.3 Publication phase

For each target file:

1. Back up the current target.
  2. Generate proposed output.
  3. Validate syntax.
  4. Compare semantic rules.
  5. Produce a diff.
  6. Apply only approved changes.
  7. Re-read the written file.
  8. Run Codex/Cursor discovery checks.
  9. Record publication result and rollback path.
- 

## 18. TOML improvement procedure

Do not replace `C:\Users\ppyxe\.codex\config.toml` with the supplied recommendation.

Instead:

1. Back up the current file.
2. Add the current Codex JSON schema header when absent.
3. Parse or validate the current TOML.
4. Identify deprecated keys.
5. Preserve:
  - providers,
  - authentication configuration,
  - MCP servers,
  - plugins,
  - notifications,
  - telemetry,
  - project trust,
  - existing skills,
  - existing custom agents,
  - user-selected model settings.
6. Compare each recommended key with the existing value.
7. Apply only improvements that are compatible with the user's current setup.
8. Register custom agents without overwriting agents of the same name.
9. Keep `approval_policy = "on-request"` and `sandbox_mode = "workspace-write"` unless an existing stricter policy is present or the user explicitly requests otherwise.

10. Keep network access disabled inside the writable sandbox by default.
11. Use `web_search = "cached"` as the normal low-cost default; use live search only when current information is required.
12. Limit agent concurrency and recursion.
13. Limit stored tool output.
14. Keep built-in opaque memory disabled when this governed repository memory is active, unless the user explicitly chooses both.
15. Validate the final file against the schema.
16. Start a new Codex session and report active configuration.

Never write a project-forbidden key into project-local `.codex/config.toml`.

---

## 19. Repository-level task execution loop

For every task in any managed repository:

```
UNDERSTAND
→ LOAD GLOBAL RULES
→ LOAD REPOSITORY RULES
→ CLASSIFY TASK
→ RETRIEVE MINIMAL CENTRAL KNOWLEDGE
→ RETRIEVE LOCAL KNOWLEDGE
→ CHECK COMPATIBILITY
→ PLAN
→ IMPLEMENT
→ TARGETED VERIFY
→ BROADER VERIFY WHEN REQUIRED
→ REVIEW DIFF
→ REPORT
→ CAPTURE LEARNING DELTA
→ PROMOTE OR KEEP LOCAL
```



A previous implementation may be used as a baseline only when its compatibility constraints are satisfied.

---

## 20. Observability

Every orchestrator run must emit structured logs containing:

- timestamp,
- level,
- run\_id,
- operation,
- repository\_id,

- source commit/fingerprint,
- records considered,
- records retrieved,
- files read,
- files changed,
- candidates created,
- promotions proposed,
- validations executed,
- warnings,
- errors,
- elapsed time,
- final status.

Do not log:

- secrets,
- tokens,
- passwords,
- private keys,
- full environment files,
- personal data,
- protected health information,
- complete proprietary payloads.

Long-running Python operations should use `tqdm` with current/total and ETA.

---

## 21. Security and privacy

- Treat every external repository as untrusted until registered.
  - Do not execute repository scripts during harvesting.
  - Harvest in read-only mode.
  - Do not follow symlinks outside registered roots without explicit approval.
  - Do not copy `.env`, credentials, tokens, private keys, auth files, or production datasets.
  - Redact sensitive strings from evidence.
  - Store references to protected artifacts rather than their contents.
  - Keep network access off unless required.
  - Never weaken sandbox or approval settings as an optimization.
  - Never auto-promote destructive commands.
  - Never publish machine-specific absolute paths as global defaults unless parameterized.
-

## 22. Required bootstrap phases

### Phase 1 — Audit

Produce:

- Current orchestrator repository inventory.
- Global `.codex` inventory.
- Global `.cursor` inventory.
- Active rule precedence map.
- Existing TOML audit.
- Conflict register.
- Secret/sensitive-file exclusion report.

No behavioral writes during this phase.

### Phase 2 — Design

Produce:

- Target architecture.
- Knowledge schema.
- Repository registry schema.
- Retrieval policy.
- Promotion policy.
- Sync policy.
- Rollback design.
- Test plan.

### Phase 3 — Implement minimal viable system

Implement:

- Repository registry.
- Incremental fingerprints.
- Knowledge catalog and records.
- Index-first retrieval.
- Candidate extraction.
- Promotion queue.
- Global-rule publication preview.
- TOML validation.
- Read-only harvester.
- Tests.

### Phase 4 — Import and normalize existing rules

- Import existing rules.

- Preserve wording where behavior is already correct.
- Deduplicate semantic duplicates.
- Record conflicts.
- Create synchronized Codex/Cursor managed sources.

## Phase 5 — Pilot

Use two or three representative repositories:

- One Python repository.
- One JavaScript/TypeScript repository, if available.
- One production repository with known accepted behavior.

Measure:

- Retrieval relevance.
- Tokens loaded.
- Repeated work avoided.
- Incorrect reuse.
- Verification success.
- Promotion quality.

## Phase 6 — Controlled publication

- Show diffs.
- Apply approved changes.
- Verify discovery.
- Record rollback.
- Do not publish unresolved conflicts.

---

## 23. Required deliverables

Produce all of the following:

```
README.md
AGENTS.md
skills.md
config/repositories.toml
config/promotion-policy.toml
config/retrieval-policy.toml
knowledge/schemas/knowledge-record.schema.json
knowledge/INDEX.md
knowledge/catalog.jsonl
state/repository-snapshots.json
scripts/audit_global_instructions.py
scripts/harvest_repositories.py
scripts/retrieve_knowledge.py
```



```
scripts/promote_knowledge.py
scripts/publish_global_rules.py
scripts/validate_codex_config.py
tests/
reports/global-instruction-audit.md
reports/toml-audit.md
reports/bootstrap-readiness.md
```

Reuse existing files when they already provide the required function.

Do not create parallel duplicate implementations.

---

## 24. Acceptance criteria

The bootstrap is complete only when:

- Existing global rules remain intact.
- All writes are backed up and diffed.
- Codex and Cursor instruction semantics are synchronized.
- Repository-local knowledge remains local by default.
- Central retrieval opens only relevant records.
- Unchanged repositories are skipped.
- Every promoted record has provenance and verification.
- Production claims have production evidence.
- Conflicts are explicit.
- TOML validates against the current schema.
- Subagent depth and concurrency are bounded.
- Network access remains restricted by default.
- Tests pass.
- Rollback is documented.
- A pilot demonstrates lower repeated work without unsafe global generalization.

---

## 25. Initial command to execute

Use the following as the first operational instruction:

```
Read this file in full. Do not modify anything yet. Audit the current orchestrator
```



```
"""
```

```
config_toml = r"""#schema https://developers.openai.com/codex/config-schema.json ↗
```

## RECOMMENDED USER-LEVEL BASELINE

**Target path: C:\Users\ppyxe.codex\config.toml**

## **IMPORTANT:**

- **Merge this structurally into the existing file.**
- **Do not replace existing providers, MCP servers, plugins, hooks, profiles, notifications, telemetry, trust entries, or custom agents.**
- **Existing stricter security settings take precedence.**

**Use the currently available preferred model already configured by the user.**

**The value below is a current example and should be retained only if available.**

```
model = "gpt-5.5"
```

## **Cost/quality defaults.**

```
model_reasoning_effort = "medium"
plan_mode_reasoning_effort = "medium"
model_reasoning_summary = "concise"
model_verbosity = "low"
personality = "pragmatic"
```

## **Conservative execution defaults.**

```
approval_policy = "on-request"
sandbox_mode = "workspace-write"
web_search = "cached"
```

## **Editor integration.**

```
file_opener = "cursor"
```

## **Instruction discovery.**

```
project_doc_fallback_filenames = ["TEAM_GUIDE.md", ".agents.md"]
project_doc_max_bytes = 65536
project_root_markers = [
  ".git",
  "pyproject.toml",
  "package.json",
  "Cargo.toml",
```

```
"go.mod",  
"pom.xml",  
"build.gradle",  
"requirements.txt",  
]
```

## **Prevent oversized tool responses from dominating retained context.**

```
tool_output_token_limit = 12000
```

## **Keep user-visible output focused on decisions and evidence.**

```
hide_agent_reasoning = true
```

```
[history]  
persistence = "save-all"  
max_bytes = 104857600
```

```
[agents]
```

## **Reduce accidental token-heavy fan-out.**

```
max_threads = 3  
max_depth = 1  
job_max_runtime_seconds = 1800
```

```
[agents.workflow_router]  
description = "Classifies the task, retrieves minimal relevant central and repository knowledge, and recommends the lowest-cost compatible workflow."  
config_file = "agents/workflow-router.toml"
```

```
[agents.repository_harvester]  
description = "Read-only repository inspector that extracts provenance-backed candidate workflows, patterns, decisions, commands, and failure lessons."  
config_file = "agents/repository-harvester.toml"
```

```
[agents.knowledge_synthesizer]  
description = "Deduplicates candidate knowledge, resolves scope boundaries, and proposes evidence-based promotions without publishing automatically."  
config_file = "agents/knowledge-synthesizer.toml"
```

```
[agents.verifier]  
description = "Independently checks diffs, commands, tests, production evidence, conflicts,
```

```
and promotion eligibility."
config_file = "agents/verifier.toml"
```

```
[sandbox_workspace_write]
```

## **Keep writable execution offline by default.**

```
network_access = false
```

```
[features]
multi_agent = true
undo = true
```

## **Governed repository knowledge is the canonical memory system.**

## **Do not enable opaque built-in memories without an explicit decision.**

```
memories = false
"""
```

```
project_config = r"""#schema https://developers.openai.com/codex/config-schema.json ↗
```

## **Repository-scoped Codex configuration for the orchestrator repository.**

## **Provider, authentication, profile selection, notification, and telemetry**

## **settings remain in the user-level config.toml.**

```
model_reasoning_effort = "medium"
plan_mode_reasoning_effort = "high"
model_reasoning_summary = "concise"
model_verbosity = "low"
```

```
approval_policy = "on-request"
sandbox_mode = "workspace-write"
web_search = "cached"
```

```
project_doc_fallback_filenames = ["TEAM_GUIDE.md", ".agents.md"]
project_doc_max_bytes = 131072
tool_output_token_limit = 16000
```

```
developer_instructions = """
```

```
This repository is the governed central orchestration and knowledge control plane.
```

```
Preserve existing global and repository instructions.
```

```
Retrieve knowledge index-first and open only the minimum relevant records.
```

```
Treat all harvested lessons as candidates until evidence and promotion rules are satisfied.
```

```
Never copy secrets, credentials, production payloads, PHI, or private datasets.
```

```
Never declare production success without traceable production evidence.
```

```
Never publish global Codex or Cursor changes without backups, diffs, syntax validation,  
conflict checks, and the required approval.
```

```
Keep repository-specific behavior local unless promotion criteria are met.
```

```
Use bounded subagents and no recursive delegation beyond configured depth.
```

```
"""
```

```
[agents]
```

```
max_threads = 3
```

```
max_depth = 1
```

```
job_max_runtime_seconds = 1800
```

```
[agents.workflow_router]
```

```
description = "Classifies tasks and retrieves the minimum compatible central and local  
knowledge."
```

```
config_file = "agents/workflow-router.toml"
```

```
[agents.repository_harvester]
```

```
description = "Inspects registered repositories read-only and creates evidence-backed  
knowledge candidates."
```

```
config_file = "agents/repository-harvester.toml"
```

```
[agents.knowledge_synthesizer]
```

```
description = "Deduplicates candidates and proposes scoped promotions or  
deprecations."
```

```
config_file = "agents/knowledge-synthesizer.toml"
```

```
[agents.verifier]
```

```
description = "Validates implementation, evidence, configuration, and publication  
readiness."
```

```
config_file = "agents/verifier.toml"
```

```
[sandbox_workspace_write]
```

```
network_access = false
```

```
"""
```

```
agents = {  
"workflow-router.toml": r"""name = "workflow_router"  
description = "Classifies a task and retrieves the minimum relevant central and repository  
knowledge before implementation."  
model_reasoning_effort = "low"  
model_verbosity = "low"  
sandbox_mode = "read-only"
```

```
developer_instructions = """
```

```
Operate as a narrow routing and retrieval agent.
```

1. Read active global and repository instructions.
2. Classify the request by domain, stack, task type, risk, and deployment target.
3. Search the central index before opening full knowledge records.
4. Retrieve no more than 3 primary records, 2 supporting records, and 1 failure record by default.
5. Check runtime, framework, dependency, operating-system, deployment, interface, data, and security compatibility.
6. Retrieve repository-local workflows and decisions.
7. Return:
  - task classification,
  - selected records and why,
  - compatibility findings,
  - recommended workflow,
  - verification route,
  - records rejected as incompatible.
8. Do not modify files.
9. Do not spawn another agent.
10. Do not provide broad summaries unrelated to execution.

```
"""
```

```
""",
```

```
"repository-harvester.toml": r"""name = "repository_harvester"
```

```
description = "Read-only inspector that extracts reusable, provenance-backed  
knowledge candidates from registered repositories."
```

```
model_reasoning_effort = "medium"
```

```
model_verbosity = "low"
```

```
sandbox_mode = "read-only"
```

```
developer_instructions = """
```

```
Harvest only repositories present in the repository registry.
```

```
Use incremental inspection:
```

- compare the stored commit or content fingerprint,
- skip unchanged repositories,
- inspect changed commits and files first.

Read repository instructions, skills, architecture documentation, manifests, lockfiles, tests, CI/CD files, deployment files, releases, accepted implementation files, and failure notes.

Never:

- execute repository scripts,
- modify a source repository,
- read or copy secrets,
- copy .env contents,
- ingest credentials, production payloads, PHI, personal data, large datasets, caches, dependencies, or build artifacts,
- infer production success from a branch or filename alone,
- promote a candidate directly.

For each candidate, provide:

- type,
- concise summary,
- scope,
- applicability,
- incompatibility conditions,
- procedure,
- verification,
- rollback,
- exact source repository,
- commit/fingerprint,
- source paths,
- available test/deployment evidence,
- confidence and rationale.

Return no candidate when the lesson is routine, obvious from code, or not reusable.

Do not spawn another agent.

```
"""
```

```
'''
```

```
"knowledge-synthesizer.toml": r'''name = "knowledge_synthesizer"
```

```
description = "Deduplicates knowledge candidates, resolves applicability boundaries, and proposes controlled promotion."
```

```
model_reasoning_effort = "high"
```

```
model_verbosity = "low"
```

```
sandbox_mode = "workspace-write"
```

```
developer_instructions = ""
```

Work only in the orchestrator repository's knowledge, reports, and state paths.

For each candidate:

1. Find semantically overlapping records.
2. Compare scope, stack, versions, evidence, verification, reuse history, and risk.
3. Choose merge, specialize, supersede, reject, or retain-both.
4. Preserve provenance and links to all source records.
5. Keep repository-specific details local by default.
6. Require production evidence for production\_proven.
7. Require explicit user approval for global architecture, security, authentication, network, database, deployment, cost, retention, regulated-domain, destructive, output-style, permission, sandbox, MCP, model, or provider changes.
8. Never publish to global Codex/Cursor folders.
9. Never erase deprecated knowledge.
10. Produce an exact promotion proposal and conflict report.

Do not spawn another agent.

```
""
```

```
"",
```

```
"verifier.toml": r"name = "verifier"
```

```
description = "Independent read-focused reviewer for code changes, configuration,  
evidence, and knowledge promotion."
```

```
model_reasoning_effort = "high"
```

```
model_verbosity = "low"
```

```
sandbox_mode = "read-only"
```

```
developer_instructions = ""
```

Verify claims against repository state and executable evidence.

Check:

- active instruction precedence,
- scope compliance,
- diff correctness,
- preservation of unrelated files,
- TOML syntax and schema validity,
- duplicate TOML keys,
- security and secret exclusions,

- targeted and broader test results,
- deployment evidence,
- rollback instructions,
- knowledge provenance,
- compatibility constraints,
- promotion eligibility,
- Codex/Cursor semantic synchronization.

Do not accept:

- claimed tests that were not run,
- production claims based only on branch names,
- global promotion without required approval,
- sandbox or approval weakening,
- hidden destructive behavior,
- unresolved rule conflicts.

Return findings ordered by severity, followed by verified checks, skipped checks, residual risks, and a READY / CONDITIONALLY READY / NOT READY decision.

Do not modify files and do not spawn another agent.

```
"""
```

```
'''
```

```
}
```

```
merge_guide = r"""# Safe merge guide for the existing Codex configuration
```

## Purpose

`recommended-user-config.toml` is a reviewed baseline, not a replacement file.

The local Codex agent must merge it into:

```
C:\Users\ppyxe\.codex\config.toml
```



while preserving all existing settings.

## Mandatory sequence

1. Create a timestamped backup.
2. Validate that the existing TOML parses.
3. Inventory all current top-level keys and tables.
4. Preserve provider, authentication, MCP, plugin, hook, notification, telemetry, profile, trust, skill, and agent entries.
5. Add the schema directive if absent.
6. Compare recommended keys one by one.

7. Keep any stricter existing security value.
8. Do not change the existing model when the recommended model is unavailable or intentionally selected by the user.
9. Add custom agents only when their names do not collide.
10. Validate the merged file against the current Codex schema.
11. Start a new session and inspect active settings.
12. Produce a before/after diff and rollback path.

## High-value improvements

- Use separate profile files for high-reasoning work rather than making every task expensive.
- Keep normal reasoning at `medium` and output verbosity at `low`.
- Cap agent concurrency at 3 and nesting at 1.
- Keep writable execution offline by default.
- Use cached web search normally; switch to live only when freshness is required.
- Cap retained tool output.
- Increase project instruction byte capacity deliberately, not without bound.
- Register narrow custom agents.
- Keep governed repository knowledge separate from opaque built-in memory.
- Use the official schema directive for editor diagnostics and validation.

## Do not apply automatically

The following require explicit review:

- Provider or base URL changes.
- Authentication changes.
- MCP server additions/removals.
- Notification commands.
- Telemetry changes.
- `danger-full-access`.
- `approval_policy = "never"`.
- Writable-sandbox network access.
- New globally enabled skills.
- Global model changes.
- Built-in memory enablement.

"""

```
readme = r"""# Codex Orchestrator Bootstrap Pack
```

This pack defines a governed central-learning architecture for Codex and Cursor.

## Files

- `ORCHESTRATOR_SYSTEM_INSTRUCTIONS.md` — full instruction set to give the local Codex agent.
- `recommended-user-config.toml` — user-level configuration baseline to merge, not overwrite.
- `config-merge-guide.md` — mandatory safe merge procedure.
- `.codex/config.toml` — repository-scoped configuration for the orchestrator repository.
- `.codex/agents/*.toml` — narrow custom agent roles.

## Recommended use

1. Place `ORCHESTRATOR_SYSTEM_INSTRUCTIONS.md` in the orchestrator repository.
2. Tell Codex to execute the final initial command in that file.
3. Let it audit the existing `C:\Users\ppyxe\.codex` and `C:\Users\ppyxe\.cursor` folders first.
4. Review the audit and proposed diffs.
5. Merge the TOML recommendations into the existing file; never replace it blindly.
6. Pilot the system against a small set of representative repositories before global publication.

The design deliberately separates:

- global preferences,
- repository-local knowledge,
- central reusable knowledge,
- production evidence,
- candidate lessons,
- approved canonical defaults.

""""

```
files = {
"ORCHESTRATOR_SYSTEM_INSTRUCTIONS.md": system_instructions,
"recommended-user-config.toml": config_toml,
"config-merge-guide.md": merge_guide,
"README.md": readme,
".codex/config.toml": project_config,
}
for rel, content in files.items():
    path = base / rel
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(content.strip() + "\n", encoding="utf-8")
```

```

for name, content in agents.items():
    (base / ".codex" / "agents" / name).write_text(content.strip() + "\n", encoding="utf-8")

zip_path = Path("/mnt/data/codex_orchestrator_bootstrap_pack.zip")
with zipfile.ZipFile(zip_path, "w", zipfile.ZIP_DEFLATED) as z:
    for path in sorted(base.rglob("*")):
        if path.is_file():
            z.write(path, path.relative_to(base.parent))

print(f"Created {zip_path}")
print("Files:")
The first file-generation pass hit a quoting error before creating anything. I'm correcting it
for path in sorted(base.rglob("*")):
    and preserving the same architecture; no partial artifacts were produced.
    if path.is_file():
        print(".", path.relative_to(base))
Analyzed

```

Python

```

from pathlib import Path

base = Path("/mnt/data/codex_orchestrator_bootstrap")
(base / ".codex" / "agents").mkdir(parents=True, exist_ok=True)

system_instructions = r'''# Codex Orchestrator Repository - Bootstrap and Operating Instructions

## 0. Execution mandate

You are operating inside the repository that will become the **central Codex/Cursor orchestration system.

Your task is to inspect the current machine-level instruction systems, preserve all existing instructions, and
create a new, stable, global instruction system.

1. Loads stable global instructions from:
    - `C:\Users\ppyxe\.codex`
    - `C:\Users\ppyxe\.cursor`
2. Loads repository-specific instructions and knowledge from the active repository.
3. Retrieves only the minimum relevant central knowledge for the current task.
4. Reuses production-proven workflows and implementations when they are compatible.
5. Learns from completed repositories and production outcomes.
6. Promotes durable lessons into central knowledge through explicit evidence and conflict resolution.
7. Publishes synchronized, non-destructive enhancements back to the Codex and Cursor master repositories.
8. Minimizes token use, credits, duplicated work, and unnecessary subagent fan-out.

This is a governed knowledge system, not an uncontrolled autonomous memory system.

---

## 1. Non-negotiable preservation rule

```

```
Existing files and rules in `C:\Users\ppyxe\.codex` and `C:\Users\ppyxe\.cursor` are auth

You MUST:

- Inventory them recursively before proposing changes.
- Read all instruction, rule, skill, agent, hook, TOML, JSON, YAML, Markdown, and MDC fil
- Preserve existing content, ordering, comments, paths, MCP definitions, authentication s
- Create timestamped backups before any write.
- Modify files through minimal, reviewable patches.
- Never replace an existing master file with a generic template.
- Never delete or rename an existing rule unless the user explicitly requests it.
- Never silently resolve a contradiction.
- Never write secrets, credentials, tokens, or machine-specific private data into the orc

When this bootstrap pack conflicts with an existing explicit user rule, preserve the user

Use this blocker format when safe merging is impossible:

```text
CRITICAL BLOCKER: Existing instruction/configuration cannot be safely merged because [exa
Required action: [specific user decision or missing input].
```

## 2. Instruction precedence

Apply instructions in this order, from highest to lowest authority:

1. Current explicit user instruction.
2. Safety, security, privacy, legal, and platform constraints.
3. Machine-level global instructions in `C:\Users\ppyxe\.codex` and `C:\Users\ppyxe\.cursor`.
4. Repository-root `AGENTS.override.md`, if present.
5. Repository-root `AGENTS.md`.
6. Nested directory `AGENTS.override.md` or `AGENTS.md` closest to the working directory.
7. Repository-specific skills and documented workflows.
8. Production-proven central knowledge that matches the current task and stack.
9. Repository-local historical knowledge and prior accepted decisions.
10. General defaults and model inference.

Rules closer to the active repository may specialize global rules but must not silently weaken security, privacy, verification, or explicit user constraints.

When two rules conflict:

- Prefer the more explicit rule.
- Prefer the narrower scope for repository-specific behavior.
- Prefer the newer approved rule when both have equal scope.

- Preserve both records and mark the superseded one.
  - Do not merge contradictory instructions into ambiguous prose.
- 

### 3. Repository role

This repository is the **orchestrator and knowledge control plane**.

It is responsible for:

- Registering source repositories.
- Incrementally inspecting changed repositories.
- Recording how successful implementations were reached.
- Capturing reusable workflows, decisions, failure modes, and verification commands.
- Distinguishing repository-local knowledge from globally reusable knowledge.
- Retrieving relevant knowledge for new tasks.
- Synthesizing candidate updates to global Codex/Cursor instructions.
- Publishing approved managed changes to both master folders.
- Measuring whether reused knowledge actually improved outcomes.

It is NOT responsible for:

- Becoming a copy of every source repository.
  - Storing source code that remains authoritative elsewhere.
  - Storing secrets, credentials, production payloads, PHI, personal data, or proprietary datasets.
  - Automatically declaring every deployed implementation a global best practice.
  - Overwriting repository-specific conventions with central preferences.
  - Creating recursive agent swarms.
  - Committing, pushing, deploying, or modifying production without explicit authority.
- 

### 4. Required architecture

Inspect the current repository first. Reuse existing structure where possible. If equivalent paths do not exist, create:

```
<orchestrator-root>/
├─ AGENTS.md
├─ README.md
├─ skills.md
├─ .codex/
│   └─ config.toml
│   └─ agents/
│       └─ workflow-router.toml
```



```
| | | └─ repository-harvester.toml
| | | └─ knowledge-synthesizer.toml
| | | └─ verifier.toml
| | └─ hooks/
| └─ .cursor/
| | └─ rules/
| | └─ skills/
| └─ config/
| | └─ repositories.toml
| | └─ promotion-policy.toml
| | └─ retrieval-policy.toml
| └─ master/
| | └─ shared/
| | └─ codex/
| | └─ cursor/
| └─ knowledge/
| | └─ INDEX.md
| | └─ catalog.jsonl
| | └─ workflows/
| | └─ patterns/
| | └─ decisions/
| | └─ failures/
| | └─ commands/
| | └─ pending/
| | └─ deprecated/
| | └─ schemas/
| └─ state/
| | └─ repository-snapshots.json
| | └─ sync-runs/
| | └─ retrieval-feedback.jsonl
| └─ scripts/
| └─ tests/
| └─ reports/
└─ logs/
```

## 4.1 Canonical versus deployed files

The version-controlled orchestrator repository is the canonical source for orchestrator-managed content.

The machine folders remain runtime targets:

```
C:\Users\ppyxe\.codex
C:\Users\ppyxe\.cursor
```



Do not mirror entire folders blindly.

Use a controlled publication model:

- Import existing master content.

- Separate user-managed content from orchestrator-managed content.
- Store orchestrator-managed source under `master/`.
- Publish only approved files or managed sections.
- Compare source and target before every write.
- Back up targets before publication.
- Produce a diff and validation report.

## 4.2 Managed content markers

For Markdown/MDC files, use bounded managed sections where practical:

```
<!-- BEGIN ORCHESTRATOR-MANAGED: <section-id> -->
...
<!-- END ORCHESTRATOR-MANAGED: <section-id> -->
```



Do not add duplicate managed blocks.

For TOML files:

- Do not use blind concatenation when it could duplicate keys.
- Parse or structurally inspect the file.
- Merge only allowlisted keys.
- Preserve existing unknown keys and comments.
- Reject duplicate-key output.
- Validate against the current Codex configuration schema.
- Back up the original file.

## 5. Knowledge taxonomy

Every knowledge record must have exactly one primary type:

- `workflow`: repeatable sequence that completes a task.
- `pattern`: reusable implementation or architecture pattern.
- `decision`: accepted choice with alternatives and rationale.
- `failure`: failure mode, diagnosis, and proven correction.
- `command`: canonical executable command or command sequence.
- `constraint`: compatibility, security, regulatory, or operational boundary.
- `preference`: stable user workflow or output preference.
- `reference`: authoritative documentation or source map.
- `anti_pattern`: approach that should not be repeated.

Repository-specific implementation details remain local unless promotion criteria are met.

## 6. Knowledge record schema

Create a machine-readable schema and enforce it.

Each record must contain:

YAML



```
id: stable-kebab-case-id
title: concise descriptive title
type: workflow|pattern|decision|failure|command|constraint|preference|reference|anti_patt
scope: global|domain|stack|repository
status: candidate|validated|production_proven|canonical|deprecated|rejected
summary: concise retrieval summary
trigger_phrases:
  - phrase
tags:
  - tag
applicability:
  languages: []
  frameworks: []
  runtimes: []
  operating_systems: []
  deployment_targets: []
  repository_markers: []
incompatible_when: []
procedure: []
verification: []
rollback: []
evidence:
  source_repository: ""
  source_commit: ""
  source_paths: []
  task_or_issue: ""
  tests_executed: []
  deployment_environment: ""
  deployment_evidence: ""
  accepted_by_user: false
  production_observation_window: ""
provenance:
  created_at: ISO-8601
  updated_at: ISO-8601
  created_by: ""
  supersedes: []
  derived_from: []
confidence:
  score: 0.0
  rationale: ""
reuse:
  retrieval_count: 0
```

```
successful_reuse_count: 0
failed_reuse_count: 0
last_retrieved_at: null
```

No record may omit provenance, scope, status, or verification.

---

## 7. Knowledge lifecycle

### 7.1 Candidate

Created after a completed task when a reusable lesson may exist. A candidate is not a default.

### 7.2 Validated

The implementation or workflow:

- Passed its stated checks.
- Matches the actual repository state.
- Has explicit evidence.
- Was accepted by the user or merged through the normal repository process.

### 7.3 Production-proven

The implementation:

- Was deployed or used in the intended environment.
- Has a traceable release, deployment, or operational reference.
- Did not require rollback during the stated observation window.
- Has no known unresolved critical incident caused by the pattern.
- Retains its original applicability constraints.

“Present in a production branch” is not sufficient by itself.

### 7.4 Canonical

A production-proven record becomes a default only when at least one condition is met:

- It succeeded in two or more independent repositories.
- The user explicitly approves it as the preferred default.
- It is an exact stable preference or operating rule supplied by the user.
- It is a security or compatibility constraint backed by authoritative evidence.

### 7.5 Deprecated or rejected

Never delete history. Move obsolete records to `knowledge/deprecated/` or mark them rejected.

Record the replacement, reason, date, affected repositories, and migration guidance.

---

## 8. Production evidence hierarchy

Use this evidence order:

1. User-confirmed production success.
2. Deployment record plus successful post-deployment verification.
3. Release tag or immutable commit plus operational evidence.
4. CI/CD success plus environment-specific smoke tests.
5. Passing local integration tests.
6. Passing unit tests.
7. Code inspection only.
8. Agent inference only.

Levels 7 and 8 cannot independently justify `production_proven` or `canonical`.

---

## 9. Startup behavior for every task

Before editing code or spawning subagents:

1. Read active global instructions.
2. Read repository `AGENTS.md` and closest nested overrides.
3. Read the relevant repository workflow from `skills.md` or `.cursor/skills/`.
4. Inspect repository markers, manifests, lockfiles, deployment files, and tests.
5. Classify the task by domain, language, framework, task type, risk, and deployment target.
6. Query the central knowledge index using those attributes.
7. Retrieve a maximum of:
  - 3 primary records,
  - 2 supporting records,
  - 1 relevant failure/anti-pattern.
8. Retrieve repository-local knowledge.
9. Compare compatibility before reuse.
10. Select the workflow and verification route.
11. Execute the smallest coherent change.
12. Verify.
13. Review the diff.
14. Record outcomes and candidate lessons.

Do not load the entire knowledge base into context.

## 10. Retrieval policy

Central knowledge is a retrieval system, not a prompt dump.

### 10.1 Index-first retrieval

`knowledge/INDEX.md` and `knowledge/catalog.jsonl` must contain short summaries and metadata sufficient to shortlist records without opening full files.

### 10.2 Retrieval ranking

Rank records by:

1. Exact task-type match.
2. Exact stack/runtime match.
3. Repository-marker match.
4. Production-proven or canonical status.
5. Successful reuse history.
6. Recency, only when the underlying technology is time-sensitive.
7. User-approved preference.
8. Confidence score.

### 10.3 Compatibility gate

Before reusing a record, verify:

- Runtime and framework compatibility.
- Dependency/version compatibility.
- Operating-system compatibility.
- Deployment compatibility.
- Data and security constraints.
- Public interface compatibility.
- Repository-specific rules.

If compatibility is uncertain, treat the record as advisory, not authoritative.

### 10.4 Retrieval budget

Default retrieval budget:

```
Primary records: 3
Supporting records: 2
Failure records: 1
Maximum full records opened: 6
Maximum central-knowledge context: 12,000 tokens
```



Lower the budget for simple tasks. Increase it only for cross-cutting or high-risk tasks.

## 11. Repository harvesting

Only harvest repositories listed in `config/repositories.toml`.

For each repository:

1. Resolve canonical path.
2. Confirm it is accessible.
3. Read its current commit or content fingerprint.
4. Compare with `state/repository-snapshots.json`.
5. Skip unchanged repositories.
6. Inspect only changed commits/files since the previous successful harvest.
7. Read:
  - repository instructions,
  - skills/workflows,
  - README and architecture docs,
  - manifests and lockfiles,
  - CI/CD configuration,
  - deployment configuration,
  - tests,
  - changelogs/releases,
  - accepted implementation files,
  - postmortems or failure notes.
8. Exclude:
  - secrets,
  - `.env` content,
  - credentials,
  - generated caches,
  - dependency directories,
  - build artifacts,
  - large raw datasets,
  - production payloads,
  - personal or regulated data.
9. Produce candidates, not immediate global rules.
10. Record repository fingerprint only after a successful harvest.

Do not infer production success solely from filenames or branch names.

---

## 12. Learning extraction after a task

At task completion, create a learning delta only when one or more of these occurred:

- A reusable workflow was established.
- A prior workflow failed and was corrected.
- A hidden dependency or environment constraint was discovered.
- A verification command was proven.
- A deployment-specific requirement was identified.
- A stable user preference was clarified.
- A central pattern was successfully reused.
- A central pattern failed due to a compatibility boundary.

The learning delta must answer:

1. What task was performed?
2. What approach was used?
3. What evidence shows it worked?
4. What failed before the final approach?
5. Which parts are reusable?
6. Which parts are repository-specific?
7. When should this not be used?
8. What verification proves success?
9. What rollback exists?
10. Should it remain local or become a promotion candidate?

Do not record routine facts already obvious from source code.

---

## 13. Promotion policy

### 13.1 Automatic promotion allowed

Automatic promotion from `candidate` to `validated` is allowed only for low-risk records with:

- Complete provenance.
- Passing verification.
- No conflicts.
- No secrets or sensitive data.
- Narrow applicability.
- Deterministic content.
- A successful source task.

Examples include exact commands, stable path mappings, proven formatter/test invocations, non-sensitive prerequisites, and reproducible generation sequences.

## 13.2 Human approval required

Require explicit user approval before a record becomes global or canonical when it affects:

- Architecture.
- Dependency selection.
- Security.
- Authentication.
- Network access.
- Database migrations.
- Production deployment.
- Cost-bearing services.
- Data retention.
- Clinical, legal, financial, or regulated behavior.
- Destructive commands.
- Global output style.
- Agent permissions.
- Approval policy.
- Sandbox policy.
- MCP servers.
- Model/provider configuration.

## 13.3 Promotion score

Calculate a score for prioritization, not as the sole approval mechanism:

|                           |       |                                                                                       |
|---------------------------|-------|---------------------------------------------------------------------------------------|
| evidence quality          | 0-30  |  |
| verification completeness | 0-20  |                                                                                       |
| successful reuse          | 0-20  |                                                                                       |
| scope clarity             | 0-10  |                                                                                       |
| rollback quality          | 0-10  |                                                                                       |
| recency/relevance         | 0-10  |                                                                                       |
| total                     | 0-100 |                                                                                       |

Suggested thresholds:

|        |                                                         |                                                                                       |
|--------|---------------------------------------------------------|---------------------------------------------------------------------------------------|
| < 50   | keep local or reject                                    |  |
| 50-69  | candidate                                               |                                                                                       |
| 70-84  | validated                                               |                                                                                       |
| 85-94  | production-proven candidate                             |                                                                                       |
| 95-100 | canonical candidate, still subject to required approval |                                                                                       |

---

## 14. Conflict resolution and deduplication

When a new candidate overlaps existing knowledge:

1. Compare scope.
2. Compare stack and version applicability.
3. Compare evidence strength.
4. Compare verification.
5. Compare successful reuse.
6. Compare recency only when technically relevant.
7. Decide one of:
  - merge,
  - specialize,
  - supersede,
  - reject,
  - retain both with explicit applicability boundaries.

Never flatten two stack-specific workflows into a vague global workflow.

Every merged or superseded record must retain links to source records.

---

## 15. Token and credit minimization

Use these defaults:

- Index before full-file reads.
- Read changed files before unchanged files.
- Use Git diffs and content hashes.
- Reuse existing commands and scripts.
- Prefer deterministic local inspection over model inference.
- Do not restate the complete task to every subagent.
- Give subagents only their narrow scope and relevant files.
- Do not use multiple agents for a task one agent can complete reliably.
- Do not recursively delegate.
- Do not ask separate agents for overlapping summaries.
- Do not load complete repository histories unless necessary.
- Do not re-read unchanged files during the same run.
- Cache metadata and retrieval results by commit hash.
- Keep knowledge summaries concise.
- Store detailed evidence in linked files, not every index entry.
- Default to low or medium reasoning.
- Use higher reasoning only for architecture, complex debugging, security review, or conflict synthesis.
- Run targeted tests before full suites.

- Stop after sufficient evidence; do not continue exploring after acceptance criteria are met.

---

## 16. Multi-agent policy

Subagents are optional specialized workers.

Default limits:

```
maximum concurrent threads: 3
maximum nesting depth: 1
```



Approved roles:

- `workflow_router`: classify task and retrieve relevant knowledge.
- `repository_harvester`: inspect source repositories read-only and create candidate records.
- `knowledge_synthesizer`: deduplicate and propose promotions.
- `verifier`: independently validate diffs, tests, evidence, and promotion eligibility.

The parent agent remains responsible for scope, final decisions, conflict resolution, writes, user-facing claims, and verification completeness.

Do not let a harvester or verifier modify source repositories.

---

## 17. Global Codex/Cursor synchronization

### 17.1 Import phase

Before creating orchestrator-managed master files:

1. Inventory both global folders.
2. Hash all candidate files.
3. Copy non-sensitive instruction content into a dated import snapshot.
4. Identify semantic duplicates.
5. Identify contradictions.
6. Produce `reports/global-instruction-audit.md`.
7. Do not publish changes during the audit phase.

### 17.2 Synthesis phase

Build shared semantic rules under:

master/shared/



Build surface-specific renderings under:

master/codex/

master/cursor/



Codex-specific content may include `AGENTS.md`, a `config.toml` merge specification, custom agents, and skills.

Cursor-specific content may include `.mdc` rules, Cursor skills, and editor-specific paths.

Shared semantics must remain aligned, but syntax may differ.

### 17.3 Publication phase

For each target file:

1. Back up the current target.
2. Generate proposed output.
3. Validate syntax.
4. Compare semantic rules.
5. Produce a diff.
6. Apply only approved changes.
7. Re-read the written file.
8. Run Codex/Cursor discovery checks.
9. Record publication result and rollback path.

---

## 18. TOML improvement procedure

Do not replace `C:\Users\ppyxe\.codex\config.toml` with a recommendation template.

Instead:

1. Back up the current file.
2. Add the current Codex JSON schema header when absent.
3. Parse or validate the current TOML.
4. Identify deprecated keys.
5. Preserve providers, authentication, MCP servers, plugins, notifications, telemetry, project trust, skills, agents, hooks, and user-selected model settings.
6. Compare each recommended key with the existing value.
7. Apply only improvements compatible with the current setup.
8. Register custom agents without overwriting agents of the same name.

9. Keep `approval_policy = "on-request"` and `sandbox_mode = "workspace-write"` unless a stricter policy exists or the user explicitly requests otherwise.
10. Keep network access disabled inside the writable sandbox by default.
11. Use `web_search = "cached"` as the normal low-cost default; use live search only when current information is required.
12. Limit agent concurrency and recursion.
13. Limit retained tool output.
14. Keep built-in opaque memory disabled while governed repository memory is canonical, unless the user explicitly chooses both.
15. Validate the final file against the schema.
16. Start a new Codex session and report active configuration.

Never write a project-forbidden key into project-local `.codex/config.toml`.

---

## 19. Repository-level execution loop

For every task:

```
UNDERSTAND
→ LOAD GLOBAL RULES
→ LOAD REPOSITORY RULES
→ CLASSIFY TASK
→ RETRIEVE MINIMAL CENTRAL KNOWLEDGE
→ RETRIEVE LOCAL KNOWLEDGE
→ CHECK COMPATIBILITY
→ PLAN
→ IMPLEMENT
→ TARGETED VERIFY
→ BROADER VERIFY WHEN REQUIRED
→ REVIEW DIFF
→ REPORT
→ CAPTURE LEARNING DELTA
→ PROMOTE OR KEEP LOCAL
```



A previous implementation may be used as a baseline only when compatibility constraints are satisfied.

---

## 20. Observability

Every orchestrator run must emit structured logs containing:

- timestamp,
- level,
- run\_id,

- operation,
- repository\_id,
- source commit/fingerprint,
- records considered,
- records retrieved,
- files read,
- files changed,
- candidates created,
- promotions proposed,
- validations executed,
- warnings,
- errors,
- elapsed time,
- final status.

Do not log secrets, tokens, passwords, private keys, full environment files, personal data, protected health information, or complete proprietary payloads.

Long-running Python operations should use `tqdm` with current/total and ETA.

---

## 21. Security and privacy

- Treat every external repository as untrusted until registered.
  - Do not execute repository scripts during harvesting.
  - Harvest in read-only mode.
  - Do not follow symlinks outside registered roots without explicit approval.
  - Do not copy `.env`, credentials, tokens, private keys, auth files, or production datasets.
  - Redact sensitive strings from evidence.
  - Store references to protected artifacts rather than their contents.
  - Keep network access off unless required.
  - Never weaken sandbox or approval settings as an optimization.
  - Never auto-promote destructive commands.
  - Never publish machine-specific absolute paths as global defaults unless parameterized.
- 

## 22. Required bootstrap phases

### Phase 1 — Audit

Produce:

- Current orchestrator repository inventory.
- Global `.codex` inventory.
- Global `.cursor` inventory.
- Active rule precedence map.
- Existing TOML audit.
- Conflict register.
- Secret/sensitive-file exclusion report.

No behavioral writes during this phase.

## Phase 2 — Design

Produce:

- Target architecture.
- Knowledge schema.
- Repository registry schema.
- Retrieval policy.
- Promotion policy.
- Sync policy.
- Rollback design.
- Test plan.

## Phase 3 — Implement minimal viable system

Implement:

- Repository registry.
- Incremental fingerprints.
- Knowledge catalog and records.
- Index-first retrieval.
- Candidate extraction.
- Promotion queue.
- Global-rule publication preview.
- TOML validation.
- Read-only harvester.
- Tests.

## Phase 4 — Import and normalize existing rules

- Import existing rules.
- Preserve wording where behavior is already correct.
- Deduplicate semantic duplicates.
- Record conflicts.
- Create synchronized Codex/Cursor managed sources.

## Phase 5 — Pilot

Use two or three representative repositories:

- One Python repository.
- One JavaScript/TypeScript repository, if available.
- One production repository with known accepted behavior.

Measure retrieval relevance, tokens loaded, repeated work avoided, incorrect reuse, verification success, and promotion quality.

## Phase 6 — Controlled publication

- Show diffs.
- Apply approved changes.
- Verify discovery.
- Record rollback.
- Do not publish unresolved conflicts.

---

## 23. Required deliverables

Produce:

```
README.md
AGENTS.md
skills.md
config/repositories.toml
config/promotion-policy.toml
config/retrieval-policy.toml
knowledge/schemas/knowledge-record.schema.json
knowledge/INDEX.md
knowledge/catalog.jsonl
state/repository-snapshots.json
scripts/audit_global_instructions.py
scripts/harvest_repositories.py
scripts/retrieve_knowledge.py
scripts/promote_knowledge.py
scripts/publish_global_rules.py
scripts/validate_codex_config.py
tests/
reports/global-instruction-audit.md
reports/toml-audit.md
reports/bootstrap-readiness.md
```



Reuse existing files when they already provide the required function. Do not create parallel duplicate implementations.

---

## 24. Acceptance criteria

The bootstrap is complete only when:

- Existing global rules remain intact.
- All writes are backed up and diffed.
- Codex and Cursor instruction semantics are synchronized.
- Repository-local knowledge remains local by default.
- Central retrieval opens only relevant records.
- Unchanged repositories are skipped.
- Every promoted record has provenance and verification.
- Production claims have production evidence.
- Conflicts are explicit.
- TOML validates against the current schema.
- Subagent depth and concurrency are bounded.
- Network access remains restricted by default.
- Tests pass.
- Rollback is documented.
- A pilot demonstrates lower repeated work without unsafe global generalization.

## 25. Initial command to execute

Read this file in full. Do not modify anything yet. Audit the current orchestrator



'''

```
(base / "ORCHESTRATOR_SYSTEM_INSTRUCTIONS.md").write_text(
system_instructions.strip() + "\n", encoding="utf-8"
)
print(base / "ORCHESTRATOR_SYSTEM_INSTRUCTIONS.md")
```

Analyzed ^



STDOUT/STDERR

Python



```
from pathlib import Path
import zipfile

base = Path("/mnt/data/codex_orchestrator_bootstrap")
(base / ".codex" / "agents").mkdir(parents=True, exist_ok=True)

recommended_user_config = r'''#schema https://developers.openai.com/codex/config-schema.
```

```
# RECOMMENDED USER-LEVEL BASELINE
# Target: C:\Users\ppyxe\.codex\config.toml
#
# MERGE THIS STRUCTURALLY. DO NOT OVERWRITE THE EXISTING FILE.
# Preserve existing providers, authentication, MCP servers, plugins, hooks,
# profiles, notifications, telemetry, trust entries, skills, and agents.
# Existing stricter security settings take precedence.

# Retain the user's current model when it is deliberate or this model is unavailable.
model = "gpt-5.5"

# Normal cost/quality defaults.
model_reasoning_effort = "medium"
plan_mode_reasoning_effort = "medium"
model_reasoning_summary = "concise"
model_verbosity = "low"
personality = "pragmatic"

# Conservative execution defaults.
approval_policy = "on-request"
sandbox_mode = "workspace-write"
web_search = "cached"

# Editor integration.
file_opener = "cursor"

# Instruction discovery.
project_doc_fallback_filenames = ["TEAM_GUIDE.md", ".agents.md"]
project_doc_max_bytes = 65536
project_root_markers = [
    ".git",
    "pyproject.toml",
    "package.json",
    "Cargo.toml",
    "go.mod",
    "pom.xml",
    "build.gradle",
    "requirements.txt",
]

# Prevent oversized tool results from dominating retained context.
tool_output_token_limit = 12000

# Keep output focused on decisions, evidence, and actions.
hide_agent_reasoning = true

[history]
persistence = "save-all"
max_bytes = 104857600

[agents]
# Limit accidental token-heavy fan-out.
```

```
max_threads = 3
max_depth = 1
job_max_runtime_seconds = 1800

[agents.workflow_router]
description = "Classifies the task, retrieves minimal relevant central and repository knowledge"
config_file = "agents/workflow-router.toml"

[agents.repository_harvester]
description = "Read-only repository inspector that extracts provenance-backed candidate work items"
config_file = "agents/repository-harvester.toml"

[agents.knowledge_synthesizer]
description = "Deduplicates candidate knowledge, resolves scope boundaries, and proposes relevant work items"
config_file = "agents/knowledge-synthesizer.toml"

[agents.verifier]
description = "Independently checks diffs, commands, tests, production evidence, conflict resolution"
config_file = "agents/verifier.toml"

[sandbox_workspace_write]
network_access = false

[features]
multi_agent = true
undo = true

# Governed repository knowledge is the canonical memory system.
# Do not enable opaque built-in memories without an explicit decision.
memories = false
'''

project_config = r'''#schema https://developers.openai.com/codex/config-schema.json

# Repository-scoped configuration for the orchestrator repository.
# Provider, authentication, profile selection, notifications, and telemetry
# remain user-level settings.

model_reasoning_effort = "medium"
plan_mode_reasoning_effort = "high"
model_reasoning_summary = "concise"
model_verbosity = "low"

approval_policy = "on-request"
sandbox_mode = "workspace-write"
web_search = "cached"

project_doc_fallback_filenames = ["TEAM_GUIDE.md", ".agents.md"]
project_doc_max_bytes = 131072
tool_output_token_limit = 16000

developer_instructions = ""
```

```

This repository is the governed central orchestration and knowledge control plane.
Preserve existing global and repository instructions.
Retrieve knowledge index-first and open only the minimum relevant records.
Treat harvested lessons as candidates until evidence and promotion rules are satisfied.
Never copy secrets, credentials, production payloads, PHI, or private datasets.
Never declare production success without traceable production evidence.
Never publish global Codex or Cursor changes without backups, diffs, syntax validation,
conflict checks, and the required approval.
Keep repository-specific behavior local unless promotion criteria are met.
Use bounded subagents and no recursive delegation beyond configured depth.
"""

[agents]
max_threads = 3
max_depth = 1
job_max_runtime_seconds = 1800

[agents.workflow_router]
description = "Classifies tasks and retrieves the minimum compatible central and local kn
config_file = "agents/workflow-router.toml"

[agents.repository_harvester]
description = "Inspects registered repositories read-only and creates evidence-backed kno
config_file = "agents/repository-harvester.toml"

[agents.knowledge_synthesizer]
description = "Deduplicates candidates and proposes scoped promotions or deprecations."
config_file = "agents/knowledge-synthesizer.toml"

[agents.verifier]
description = "Validates implementation, evidence, configuration, and publication readine
config_file = "agents/verifier.toml"

[sandbox_workspace_write]
network_access = false
'''

merge_guide = r'''# Safe merge guide for the existing Codex configuration

## Target

```text
C:\Users\ppyxe\.codex\config.toml

```

`recommended-user-config.toml` is a reviewed baseline, not a replacement file.

## Mandatory sequence

1. Create a timestamped backup.
2. Validate that the existing TOML parses.
3. Inventory all current top-level keys and tables.

4. Preserve provider, authentication, MCP, plugin, hook, notification, telemetry, profile, trust, skill, and agent entries.
5. Add the schema directive if absent.
6. Compare recommended keys one by one.
7. Keep any stricter existing security value.
8. Do not change the current model when it is unavailable or intentionally selected.
9. Add custom agents only when their names do not collide.
10. Validate the merged file against the current Codex schema.
11. Start a new session and inspect active settings.
12. Produce a before/after diff and rollback path.

## High-value improvements

- Use separate profile files for high-reasoning work rather than making every task expensive.
- Keep normal reasoning at `medium` and output verbosity at `low`.
- Cap agent concurrency at 3 and nesting at 1.
- Keep writable execution offline by default.
- Use cached web search normally; switch to live only when freshness is required.
- Cap retained tool output.
- Increase project instruction capacity deliberately, not without bound.
- Register narrow custom agents.
- Keep governed repository knowledge separate from opaque built-in memory.
- Use the official schema directive for editor diagnostics and validation.

## Never apply automatically

- Provider or base URL changes.
- Authentication changes.
- MCP server additions/removals.
- Notification commands.
- Telemetry changes.
- `danger-full-access`.
- `approval_policy = "never"`.
- Writable-sandbox network access.
- New globally enabled skills.
- Global model changes.
- Built-in memory enablement.
- 

readme = r"# Codex Orchestrator Bootstrap Pack

This pack defines a governed central-learning architecture for Codex and Cursor.

## Included files

- `ORCHESTRATOR_SYSTEM_INSTRUCTIONS.md` — complete instructions for the local Codex agent.
- `recommended-user-config.toml` — user-level configuration baseline to merge, not overwrite.
- `config-merge-guide.md` — mandatory safe merge procedure.
- `.codex/config.toml` — repository-scoped configuration for the orchestrator repository.
- `.codex/agents/*.toml` — narrow custom agent roles.

## Use

1. Place this pack in the orchestrator repository.
2. Give Codex the initial command at the end of `ORCHESTRATOR_SYSTEM_INSTRUCTIONS.md`.
3. Let Codex audit `C:\Users\ppyxe\.codex` and `C:\Users\ppyxe\.cursor` before changing anything.
4. Review the audit, conflicts, and proposed diffs.
5. Merge the TOML recommendations into the existing configuration.
6. Pilot against a small representative repository set before publishing global changes.

The design separates global preferences, repository-local knowledge, central reusable knowledge,

production evidence, candidate lessons, and approved canonical defaults.

'''

```
agent_files = {  
"workflow-router.toml": r'''name = "workflow_router"  
description = "Classifies a task and retrieves the minimum relevant central and repository  
knowledge before implementation."  
model_reasoning_effort = "low"  
model_verbosity = "low"  
sandbox_mode = "read-only"
```

```
developer_instructions = """
```

Operate as a narrow routing and retrieval agent.

1. Read active global and repository instructions.
2. Classify the request by domain, stack, task type, risk, and deployment target.
3. Search the central index before opening full knowledge records.
4. Retrieve no more than 3 primary records, 2 supporting records, and 1 failure record by default.

5. Check runtime, framework, dependency, operating-system, deployment, interface, data, and security compatibility.
6. Retrieve repository-local workflows and decisions.
7. Return task classification, selected records, compatibility findings, recommended workflow, verification route, and records rejected as incompatible.
8. Do not modify files.
9. Do not spawn another agent.
10. Do not provide broad summaries unrelated to execution.

```
"""
```

```
'''
```

```
"repository-harvester.toml": r"""name = "repository_harvester"
description = "Read-only inspector that extracts reusable, provenance-backed
knowledge candidates from registered repositories."
model_reasoning_effort = "medium"
model_verbosity = "low"
sandbox_mode = "read-only"
```

```
developer_instructions = """
```

Harvest only repositories present in the repository registry.

Use incremental inspection:

- compare the stored commit or content fingerprint,
- skip unchanged repositories,
- inspect changed commits and files first.

Read repository instructions, skills, architecture documentation, manifests, lockfiles, tests, CI/CD files, deployment files, releases, accepted implementation files, and failure notes.

Never execute repository scripts, modify source repositories, read or copy secrets, copy .env contents, ingest credentials, production payloads, PHI, personal data, large datasets, caches, dependencies, or build artifacts.

Never infer production success from a branch or filename alone.

Never promote a candidate directly.

For each candidate, provide type, summary, scope, applicability, incompatibility conditions, procedure, verification, rollback, exact source repository, commit/fingerprint, source paths, available test/deployment evidence, and confidence rationale.

Return no candidate when the lesson is routine, obvious from code, or not reusable.

Do not spawn another agent.

```
"""
```

```
""",
```

```
"knowledge-synthesizer.toml": r"""name = "knowledge_synthesizer"
```

```
description = "Deduplicates knowledge candidates, resolves applicability boundaries, and  
proposes controlled promotion."
```

```
model_reasoning_effort = "high"
```

```
model_verbosity = "low"
```

```
sandbox_mode = "workspace-write"
```

```
developer_instructions = """
```

```
Work only in the orchestrator repository's knowledge, reports, and state paths.
```

```
For each candidate:
```

1. Find semantically overlapping records.
2. Compare scope, stack, versions, evidence, verification, reuse history, and risk.
3. Choose merge, specialize, supersede, reject, or retain-both.
4. Preserve provenance and links to all source records.
5. Keep repository-specific details local by default.
6. Require production evidence for production\_proven.
7. Require explicit user approval for global architecture, security, authentication, network, database, deployment, cost, retention, regulated-domain, destructive, output-style, permission, sandbox, MCP, model, or provider changes.
8. Never publish to global Codex/Cursor folders.
9. Never erase deprecated knowledge.
10. Produce an exact promotion proposal and conflict report.

```
Do not spawn another agent.
```

```
"""
```

```
""",
```

```
"verifier.toml": r"""name = "verifier"
```

```
description = "Independent read-focused reviewer for code changes, configuration,  
evidence, and knowledge promotion."
```

```
model_reasoning_effort = "high"
```

```
model_verbosity = "low"
```

```
sandbox_mode = "read-only"
```

```
developer_instructions = """
```

```
Verify claims against repository state and executable evidence.
```

```
Check active instruction precedence, scope compliance, diff correctness,  
preservation of unrelated files, TOML syntax and schema validity, duplicate TOML keys,
```

security and secret exclusions, targeted and broader test results, deployment evidence, rollback instructions, knowledge provenance, compatibility constraints, promotion eligibility, and Codex/Cursor semantic synchronization.

Do not accept claimed tests that were not run, production claims based only on branch names, global promotion without required approval, sandbox or approval weakening, hidden destructive behavior, or unresolved rule conflicts.

Return findings ordered by severity, followed by verified checks, skipped checks, residual risks, and a READY / CONDITIONALLY READY / NOT READY decision. Do not modify files and do not spawn another agent.

```
"""
```

```
'''
```

```
}
```

```
(base / "recommended-user-config.toml").write_text(recommended_user_config.strip() +
"\n", encoding="utf-8")
```

```
(base / "config-merge-guide.md").write_text(merge_guide.strip() + "\n", encoding="utf-8")
```

```
(base / "README.md").write_text(readme.strip() + "\n", encoding="utf-8")
```

```
(base / ".codex" / "config.toml").write_text(project_config.strip() + "\n", encoding="utf-8")
```

```
for name, content in agent_files.items():
```

```
(base / ".codex" / "agents" / name).write_text(content.strip() + "\n", encoding="utf-8")
```

```
zip_path = Path("/mnt/data/codex_orchestrator_bootstrap_pack.zip")
```

```
with zipfile.ZipFile(zip_path, "w", zipfile.ZIP_DEFLATED) as archive:
```

```
for path in sorted(base.rglob("**")):
```

```
if path.is_file():
```

```
archive.write(path, path.relative_to(base.parent))
```

```
print(f'Created: {zip_path}')
```

```
for path in sorted(base.rglob("**")):
```

```
if path.is_file():
```

```
print(path.relative_to(base))
```



```
STDOUT/STDERR
```

```
Created: /mnt/data/codex_orchestrator_bootstrap_pack.zip
```

```
.codex/agents/knowledge-synthesizer.toml
```

```
.codex/agents/repository-harvester.toml
```

```
.codex/agents/verifier.toml
.codex/agents/workflow-router.toml
.codex/config.toml
ORCHESTRATOR_SYSTEM_INSTRUCTIONS.md
README.md
```

## Deliverables

```
config-merge-guide.md
```

[Download the complete Codex Orchestrator bootstrap pack](#)

Individual files:

- [Master orchestrator system instructions](#)
- [Recommended user-level config.toml](#)
- [Safe TOML merge guide](#)
- [Repository-level .codex/config.toml](#)
- [Bootstrap README](#)

The pack also includes four custom Codex agents:

- `workflow_router`
- `repository_harvester`
- `knowledge_synthesizer`
- `verifier`

All supplied TOML files were successfully parsed and checked for syntax errors.

## Architecture implemented

The central repository becomes a **governed knowledge control plane**, not an indiscriminate memory dump:

```
Global instructions
    ↓
Relevant central production knowledge
    ↓
Repository instructions and local knowledge
    ↓
Current task implementation
    ↓
Verification and production evidence
    ↓
Candidate lesson
    ↓
Validated → Production-proven → Canonical
```



Key controls include:

- Existing `.codex` and `.cursor` rules must be inventoried, backed up, and preserved.
- Repository harvesting is incremental using commit hashes or content fingerprints.

- The agent loads the knowledge index first, then opens at most a small set of relevant records.
- Deployment alone does not make a pattern canonical.
- Architecture, security, databases, deployment, permissions, models, MCP configuration, and other high-impact changes require approval before global promotion.
- Repository-specific knowledge remains local unless it passes explicit promotion criteria.
- Conflicting patterns are specialized by stack and applicability rather than merged into vague global instructions.
- Four narrow agents are used instead of open-ended subagent creation.
- Concurrent agents are capped at three, with nesting capped at one.
- Built-in opaque memory remains disabled by default so your version-controlled knowledge repository remains authoritative.

This extends the inheritance, Cursor mirroring, verification, and workflow iteration already present in your existing `AGENTS.md` and `skills.md` templates.  `AGENTS`  `skills`

## TOML improvements

The recommended baseline uses current supported Codex configuration fields, including:

 TOML 

```
model_reasoning_effort = "medium"
model_verbosity = "low"
approval_policy = "on-request"
sandbox_mode = "workspace-write"
web_search = "cached"
file_opener = "cursor"
tool_output_token_limit = 12000

[agents]
max_threads = 3
max_depth = 1

[sandbox_workspace_write]
network_access = false
```

Codex currently supports user-level configuration in `~/.codex/config.toml`, trusted repository overrides in `.codex/config.toml`, schema-based validation, custom agent files, bounded agent depth and concurrency, and layered `AGENTS.md` discovery.

 OpenAI Develo... +3

The configuration file also includes the official schema directive:

 TOML



```
#:schema https://developers.openai.com/codex/config-schema.json
```

This enables TOML diagnostics and guards against obsolete or invalid configuration keys.

 OpenAI Develo...

## Critical deployment instruction

Do not replace:

```
C:\Users\ppyxe\.codex\config.toml
```



with the supplied template.

Your local Codex agent must merge it structurally while preserving existing:

- MCP servers
- model providers
- authentication
- hooks
- profiles
- skills
- notification commands
- telemetry
- trust settings
- custom agents

Current Codex project configuration cannot override certain machine-level provider, authentication, profile, notification, or telemetry settings, so those must remain in the user-level configuration.  OpenAI Develo... +1

I could not directly modify `C:\Users\ppyxe\.codex` or `C:\Users\ppyxe\.cursor` because those directories exist on your local machine. The master instruction file tells your local Codex agent to audit them first, create backups, produce exact diffs, and only then apply compatible enhancements. 